

The Resource Divide: Data Scarcity, Compute Inequity, and Global LLM Governance

Connor Sempf and Bo Pang

School of Electrical and Computer Engineering, Georgia Institute of Technology
{csempf3, bpang42}@gatech.edu

Abstract

Large language models are positioned as universal tools, yet the resources required to train and deploy them are distributed unevenly across languages. We quantify compounding disadvantages across tokenization efficiency, memory scalability, and hardware access. Using the Llama-3 tokenizer, Hindi requires 2.83 tokens per word versus 1.15 for English, resulting in a $7.99\times$ attention cost penalty. Memory simulation across five hardware tiers shows a $40\times$ gap: RTX 4060 OOMs at 14K tokens while H100 sustains 544K. In serving scenarios with 32 concurrent users, Hindi OOMs after one turn on consumer GPUs versus three turns for English. Prefill-decode disaggregation analysis reveals decode dominates 95–99.9% of end-to-end latency. Our Composite Resource Divide Index reveals $8.1\times$ structural disparity between Swahili (0.800) and English (0.099).

I. Introduction

The rapid proliferation of large language models has centralized AI capabilities despite open-weight releases [1]. The United States controls approximately 74% of global high-end AI compute, with 160 nations classified as “compute deserts” entirely dependent on foreign infrastructure [2][3]. This concentration is institutionalized: the CHIPS Act appropriates \$52.7 billion while prohibiting semiconductor expansion in China [4], and export controls on H100/H200 GPUs create structural access asymmetries.

The AI divide extends to data. English dominates 49.7% of global web content, while Swahili—spoken by 200 million people—constitutes merely 0.0025% [5]. This creates a self-reinforcing cycle: less digital content leads to poorer language models, which reduces adoption, which discourages content creation. This paper quantifies how compute and data disparities manifest as compounding, measurable performance gaps in LLM deployment.

We present a multi-dimensional analysis combining real tokenizer benchmarks, KV cache memory simulation, serving-system analysis, and a comprehensive language ecosystem framework. Our experiments span five hardware tiers from consumer GPUs (RTX 4060) to frontier datacenter accelerators (H100), evaluating the complete inference pipeline from tokenization through multi-turn conversational serving.

II. Related Work

GovAI identifies compute as uniquely excludable, making it the primary lever for AI regulation [1]. Nations without territorial compute clusters lack “computational sovereignty” and remain perpetually dependent on foreign cloud infrastructure [2]. The Tony Blair Institute and UC Berkeley AI Policy Hub document how supply-chain concentration creates structural barriers that market mechanisms cannot self-correct [3][8].

Lundin et al. demonstrate that each additional token per word reduces downstream accuracy by 8–18 percentage points, coining the “tokenization tax” [6]. Ahuja et al. show that low-resource languages face systematic cost-performance disadvantages in LLM inference [9]. Of approximately 2,000 African languages, only 42 have meaningful LLM support [7][8].

Hooper et al. identify KV cache as the fundamental bottleneck for long-context inference [9]. vLLM’s PagedAttention reduces memory waste from 60–80% to under 4%, enabling 2–4 $\times$ throughput improvements

[10][11]. Recent work by WVA and DUET establishes that prefill (compute-bound) and decode (memory-bound) phases have fundamentally different hardware requirements, motivating phase-specific hardware disaggregation [12][13].

III. Methodology and Experiments

We employ the Llama-3 tokenizer (via HuggingFace Transformers) for real benchmarks and a KV cache memory simulation framework modeling Llama-3-8B with 4-bit AWQ quantization across five hardware tiers: H100 (80GB HBM3, 3.35 TB/s), A100 (80GB HBM2e, 2.04 TB/s), RTX 3090 (24GB GDDR6X, 936 GB/s), T4 (16GB GDDR6, 320 GB/s), and RTX 4060 (8GB GDDR6, 272 GB/s). KV cache per token is 0.125 MB for FP16 precision (formula: $2 \times 32 \text{ layers} \times 8 \text{ KV heads} \times 128 \text{ head_dim} \times 2 \text{ bytes}$).

A. Tokenization Fertility Results

Tokenization fertility—the ratio of subword tokens to source words—determines compute consumption per unit of semantic content. Table I presents fertility and attention cost multiplier (fertility²) for six languages using the Llama-3 tokenizer.

Hindi requires 2.83 tokens per word—2.46× more than English’s 1.15. Chinese achieves the most efficient encoding at 0.73 tokens per word (multiplier 0.53). Because attention scales as $O(n^2)$, Hindi incurs $7.99 \times$ attention cost versus English’s $1.32 \times$ —a $6 \times$ penalty. For the same semantic content, Hindi requires roughly six times more floating-point operations per self-attention layer.

Language	Script	Fertility	Attn. Cost
Chinese	Han	0.73	0.53
English	Latin	1.15	1.32
Spanish	Latin	1.68	2.81
Swahili	Latin	2.34	5.47
Arabic	Arabic	2.40	5.76
Hindi	Devanagari	2.83	7.99

Table I: Tokenization fertility and attention cost by language.

B. Memory Wall Simulation

KV cache grows linearly with sequence length: a 4K context at batch size 32 requires 15.4 GB (English) to 35.3 GB (Hindi). Table II shows maximum sustainable sequence lengths before OOM.

The consumer RTX 4060 (8GB) hits the memory wall at just 435 tokens for batch size 32—less than one page of text. The H100 sustains $40 \times$ more concurrent tokens (17,024). For multilingual deployments where fertility inflates token counts, the RTX 4060 threshold of ~ 435 tokens is below the length of many real-world documents.

Hardware	VRAM	Bandwidth	Max Seq (B=1)	Max Seq (B=32)
H100	80GB	3.35 TB/s	544,768	17,024
A100	80GB	2.04 TB/s	544,768	17,024
RTX 3090	24GB	936 GB/s	131,891	4,121
T4	16GB	320 GB/s	72,908	2,278
RTX 4060	8GB	272 GB/s	13,926	435

Table II: Maximum sequence length before OOM (Llama-3-8B 4-bit).

C. End-to-End Latency and Multi-Turn Serving

For an identical 100-word semantic task, end-to-end latency reveals compounding disadvantage. English on H100 completes in 0.7s; Hindi on T4 requires 18.8s—a $26\times$ gap. The Resource Divide Index ($\text{RDI} = \text{fertility}^2 / (\text{bandwidth} \times \text{VRAM})$) reaches $1,010\times$ between extremes.

Real-world LLM applications operate as multi-turn conversations where each turn appends to the KV cache. We simulate 32 concurrent users engaging in 10-turn conversations with 80 words per turn. The RTX 4060 OOMs after just 1 turn for Hindi (batch=32) versus 3 turns for English. The T4 OOMs at turn 6 for Hindi versus turn 13 for English. Only frontier GPUs sustain all 20 turns for all languages.

D. Prefill-Decode Disaggregation Analysis

LLM inference comprises two fundamentally different phases [12][13]. Prefill processes all input tokens in parallel via large matrix multiplications—compute-bound, achieving 90–95% GPU utilization on H100. Decode generates one token at a time, reading the entire KV cache from memory each step—memory-bound, with only 20–40% utilization.

Table III breaks down latency by workload type. Decode dominates 95.6–99.9% of end-to-end latency across all workloads. For a medium essay (500 input, 300 output tokens), decode consumes 99.5% of time on all GPUs. The HBM3e bandwidth at 3.35 TB/s versus GDDR6 at 272 GB/s creates a $12\times$ gap that directly governs decode throughput.

Workload	GPU	Input	Output	TTFT (ms)	Decode (ms)	Decode %
Short Q&A	H100	50	30	1.8	52.9	96.7%
Medium Essay	H100	500	300	2.4	534.7	99.5%
Long Document	H100	4000	500	10.3	959.7	98.9%
Code Gen	H100	200	800	2.0	1416.5	99.9%

Table III: Prefill vs decode latency breakdown.

Assigning prefill to H100 and decode to RTX 4060 halves serving cost (\$0.00185 vs \$0.00373 per request) but increases latency by $12\times$ (6,588 ms vs 537 ms). For cost-sensitive deployments in the Global South, this tradeoff is unavoidable.

E. Comprehensive Language Ecosystem Analysis

We synthesize six metrics into a Composite Resource Divide Index (CRDI): speaker population, internet content percentage, digital literacy proxy, estimated LLM MMLU accuracy, tokenization fertility, and Wikipedia contributor count. Table IV presents CRDI rankings for ten major languages.

Swahili scores 0.800—the worst in our sample—despite 200 million speakers, due to minuscule digital content (0.0025%), low digital literacy proxy (38%), poor LLM accuracy (45%), and high fertility (2.34). English scores 0.099—an $8.1\times$ structural advantage. The correlation analysis reveals internet content strongly correlates with Wikipedia activity ($r=0.93$) and LLM accuracy ($r=0.66$), while fertility shows strong negative correlation with LLM accuracy ($r=-0.86$).

Language	Population	Internet %	CRDI	Tier
Swahili	200M	0.0025%	0.800	Low
Bengali	230M	0.03%	0.779	Low
Arabic	450M	1.5%	0.674	Moderate
Hindi	600M	3.77%	0.668	Moderate
Spanish	500M	5.8%	0.607	High
English	380M	49.4%	0.099	High

Table IV: Composite Resource Divide Index by language.

IV. Dual Viewpoint Analysis

Viewpoint A (Bo Pang): Structural Governance Imperative

The experimental evidence supports urgent structural intervention. The compute gap is not self-correcting: a T4’s GDDR6 cannot upgrade to HBM3e, and export controls prevent frontier hardware acquisition [1][4]. The tokenization tax is equally structural—Hindi’s 2.83 tokens per word stems from English-centric BPE training on 15 trillion tokens where Hindi constitutes a negligible fraction [6].

The CRDI data reinforces this argument. Swahili’s 0.800 versus English’s 0.099 reflects systemic failure across all six measured dimensions—not merely one correctable deficiency. Policy recommendations include: (1) Sovereign AI funds financing regional compute clusters; (2) International Compute Commons agreements modeled on CERN; (3) Mandatory inclusive tokenizer design for models above threshold sizes; (4) UNESCO-expanded governance monitoring CRDI-like indices.

Viewpoint B (Connor Sempf): Data-Driven Self-Correction

While Viewpoint A correctly identifies compute barriers, it risks overemphasizing hardware at the expense of data—the root cause of poor LLM performance. The CRDI correlation analysis shows LLM accuracy strongly correlates with internet content ($r=0.66$) and negatively with fertility ($r=-0.86$). Without representative training corpora, no compute quantity bridges the gap.

Market mechanisms can work: India’s 600 million Hindi speakers represent an addressable market. XBridge and similar projects show vocabulary extension reduces fertility premiums by 40–60% [14]. The cost of curating 10 billion Swahili tokens is orders of magnitude less than building H100 clusters—and data, unlike compute, is non-excludable once released.

Synthesis

Both viewpoints identify real, non-overlapping barriers. English benefits across all dimensions simultaneously—a structural advantage no single intervention can replicate. However, compounding disadvantage implies compounding returns from coordinated action: inclusive tokenizers reduce fertility (improving compute efficiency), data commons improve model quality, and compute sharing enables deployment where markets fail. Neither governance redistribution nor data investment alone suffices; a dual strategy addressing both in parallel is required.

V. Conclusion

This paper demonstrates that the AI divide is real, quantifiable, and compounding across tokenization, memory, hardware, and data ecosystems. Key findings: (1) Hindi pays $7.99\times$ attention cost versus English’s $1.32\times$; (2) Consumer GPUs OOM at 14K tokens while frontier GPUs sustain 544K; (3) In serving scenarios, Hindi on RTX 4060 OOMs after one conversation turn versus three for English; (4) Decode dominates 95–99.9% of latency, making memory bandwidth the binding constraint; (5) The CRDI reveals $8.1\times$ structural disparity between Swahili (0.800) and English (0.099).

Moving forward requires coordinated action across three fronts. First, establish international data commons for low-resource languages, modeled on Masakhane but scaled to billion-token corpora. Second, implement regional compute-sharing agreements and sovereign AI funds, with CRDI-based metrics tracking progress. Third, mandate inclusive tokenizer design as standard practice for models above threshold sizes. The dual-viewpoint analysis shows neither markets nor governance alone suffices; only parallel investment in data equity and compute access can break the cyclic exclusion of the majority of the world’s languages from the LLM revolution.

References

- [1] GovAI, “Computing Power and the Governance of AI,” arXiv:2409.02888, 2024. [2] V. Lehdonvirta et al., “Computational Sovereignty and the AI Divide,” Proc. AIES, 2024. [3] Tony Blair Institute, “State of Compute Access,” Policy Report, 2024. [4] U.S. Congress, “CHIPS and Science Act of 2022,” Public Law 117-167, 2022. [5] W3Techs, “English accounts for 49.40% of internet content,” 2024. [6] J. Lundin et al., “The Token Tax: Systematic Bias in Multilingual Tokenization,” arXiv:2509.05486, 2025. [7] M. Abbott et al., “Masakhane—Machine Translation for Africa,” arXiv:2003.11529, 2020. [8] A. Adebara et al., “The State of LLMs for African Languages,” arXiv:2506.02280, 2025. [9] K. Ahuja et al., “Cost-Performance Optimization for Low-Resource Languages,” ACL, 2024. [10] C. Hooper et al., “KVQuant: Towards 10 Million Context Length,” UC Berkeley EECS, 2024. [11] W. Kwon et al., “vLLM: Efficient Memory Management for LLM Serving,” SOSP, 2023. [12] WVA, “A Global Optimization Control Plane for LLM Inference,” arXiv:2603.09730, 2026. [13] DUET, “Disaggregated Hybrid Mamba-Transformer LLMs,” 2025. [14] A. Nanda et al., “Reducing Tokenization Premiums for Low-Resource Languages,” arXiv:2601.13328, 2026.
-

Appendix: Compute and AI Usage Disclosure

Compute Usage: Georgia Tech AI Makerspace, ~12 hours across 3 sessions. All experiments are CPU-runnable simulations. RTX 3090 used for validation only. Total compute time: ~20 hours (development + reproduction).

AI Usage Disclosure: In accordance with Georgia Tech’s AI policy: (1) Code developed with AI assistance for boilerplate and matplotlib styling; (2) Literature research AI-assisted but manually verified; (3) Writing AI-assisted for organization and grammar; (4) Figures generated by author scripts; (5) CRDI, RDI metrics, and simulation frameworks are original contributions.

Code Repository: https://github.com/Pb314314/Fund_ML